

Solution Proposal

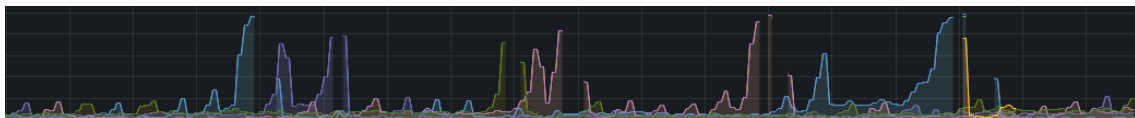
Topic	Right-Sizing Microservices
Prepared by	Ronald Thian
Prepared on	17 March 2026
Approved by	Kar Wai Cheng
Approved on	24 March 2026

Context

In March 2026, there were several incidents relating to the incompleteness of draw operations due to application pod restarting between critical operations (Lottery Results Drawing). These incidents required the direct manipulation of the database in order to rollback the state of the draw.

Problem Evaluation

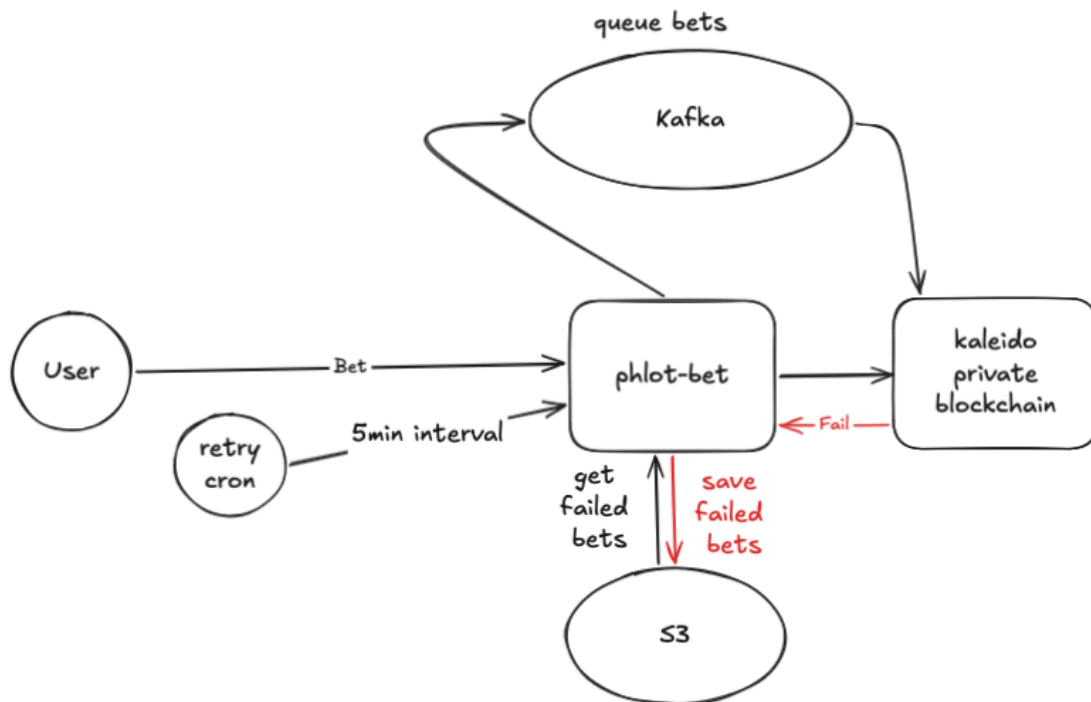
During incident investigation, it was noted that although the average utilization of each application pod remained low, there were sudden spikes of cpu and memory utilization, which eventually led to server restarts as Kubernetes found them to be unresponsive.



The prometheus chart captured at the time of incident

It was also found that the spikes lined up with the routine cron job that was set up to trigger blockchain retries. Upon further investigation, we confirmed that the spike in transaction and large backlog of failed blockchain transactions contributed to the spike and overwhelming of our application pods.

Initially designed to be a lean and safeguard process against failed blockchain transactions, a simple Kafka queue was introduced to throttle API requests to Kaleido, the private blockchain provider used in the NL System. This ensured that even if a transaction was rejected and could not make it to our blockchain, the routine cron job will ensure that repeated attempts will be made.



NL System Blockchain Failure Reattempt Design

However, as the customer base grew over the years, it slowly began to overwhelm our system and affect other key operations that we also used the same application pod, becoming a critical choke point within our system.

Temporary Preventive Measures and Hotfix

In order to provide immediate response to the incident, the Horizontal Pod Autoscaling (HPA) config was first increased to attempt to give the server more headroom to take on larger requests. However, this proved to be ineffective as failed blockchain requests grew into the numbers of 300,000.

Batched API Requests

In the early discussions of these incidents, I proposed the introduction of batched queries - instead of allowing an unlimited quantity of requests to hit Kaleido, we should chunk them into batches of requests (e.g. at most 10,000 requests at any single time). This allowed Kaleido to digest large batches in smaller chunks and prevented their fail-first mechanism to trigger. At the same time, preventing our application pods from exhausting all its resources which eventually led to kubernetes restarting it.

Update: After deploying this solution to production on 11 March 2026, the system has shown promising signs of stability. Retry attempts no longer hog the CPU utilization and server restarts due to overwhelming requests have seem to be reduced significantly.

Proposed Recommended Solution

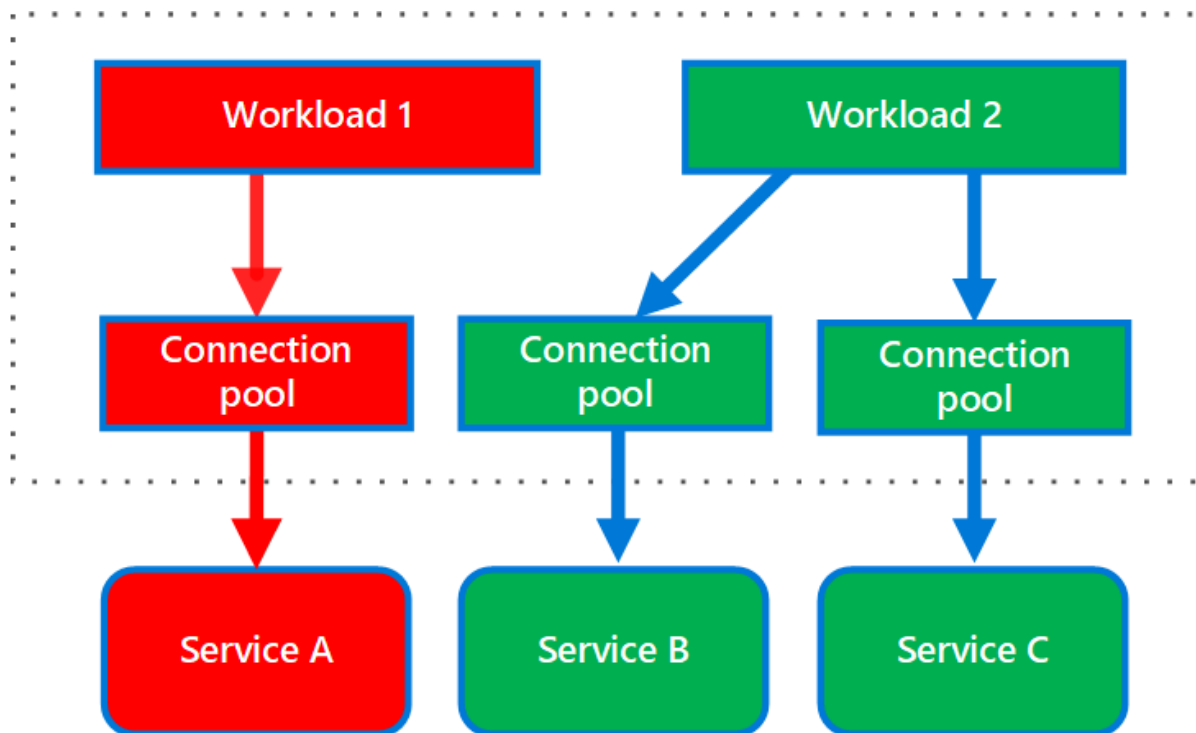
To follow up and further improve system stability, a few possible solutions could be implemented.

Phase 1: Batching Requests

As discussed earlier in the proposal, the batching of requests will act as a throttle against large batches of requests, hence, preventing the overwhelming of system services. This solution did not require infrastructure changes. As of writing, it was already deployed to production and proved to be working.

Phase 2: Bulkhead Pattern for Application Pods

In a previous solution, when the database connection pool was overwhelmed, the team implemented the bulkhead pattern [1] for connection pools. In essence, this creates groups of connection pools dedicated only for a certain functionality. This meant that the failure of a certain group of database connections would not cause the other functionalities to break, hence isolating the failure.



Similarly, this design pattern could be implemented across things like the orchestration of application pods. By dedicating application pods for critical and time sensitive functionalities, this ensures that these functions could not be affected by external factors like the blockchain retry mechanism.

These are the services that were identified to be suitable for isolation:

1. Current Main Application (No change)
2. Backoffice Operations (New)
3. Blockchain Retry Operations (New)

By isolating the Backoffice and Blockchain Retry operations, the application will be cleanly separated by its core purpose. (1) being in charge of customer facing functions like bet placing, (2) serving the core backoffice operations like draw operations and jackpot configurations and (3) reserved for the committing of data into blockchain.

Phase 3: Clean Up and Maximum Utilization

Once Phase 2 has proven to be successful, there will be much room for refactoring. As these dedicated application pods now purely serve a certain type of functions, there will be a need to code split and segregate functions, cleaning up unused source code to further reduce complexity and make it leaner to run.

Next, if an entire group of application pods is dedicated to only the reattempt of blockchain transaction, this would allow the possibility of running retries 24/7 instead of the 5 minutes cron job interval currently used. Therefore, by dedicating pods, we open up the possibility of supporting even larger amounts of requests, hence greatly improving the system's scalability.

Sources

1. Microsoft. (n.d.). *Bulkhead pattern*. In *Cloud design patterns* (Azure Architecture Center). Retrieved 12 September 2025 from <https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead>